

For now, **SpecForge** works perfectly as the internal project name. Don't spend time on branding yet. The first goal is to prove one thing:

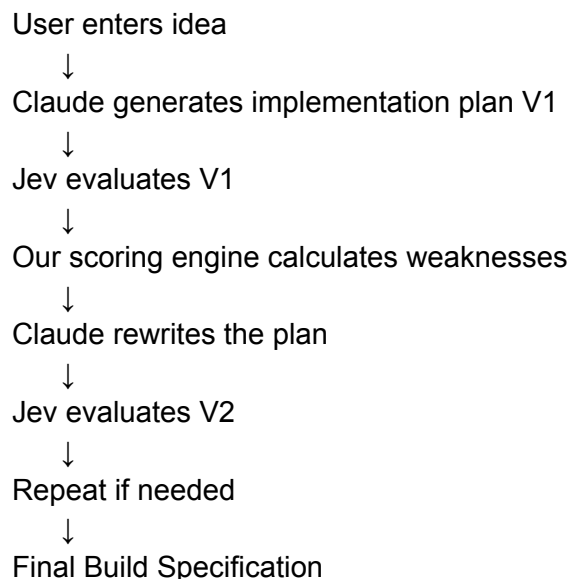
Can we take a mediocre software-building request, automatically turn it into a substantially better implementation specification, and show that the improved version actually helps an AI coding agent build better software?

That should drive the entire build.

Your Jev research supports using it as the **bounded evaluator**, not as the model that writes the plan. Jev is designed to return predefined decisions, probabilities, and rubric scores rather than open-ended plans. Pasted markdown It also specifically makes sense to combine atomic Jev judgments with deterministic code and another reasoning/generative model. Pasted markdown

The product we're building

For V1, dick123 should do exactly this:



Do **not** start with IDE integration, autonomous coding, team collaboration, billing, GitHub integration, mobile apps, or a giant dashboard.

Build the loop first.

Phase 0 — Define the brain before writing the app

This is where I would start.

Before Cursor writes a single UI component, define exactly what counts as a good software-building specification.

Your first evaluator can have roughly these dimensions:

Dimension	What we're checking
Requirement coverage	Did the plan capture what the user actually asked for?
Architecture	Is there a coherent technical architecture?
Implementation specificity	Could an agent act on this without guessing?
Data model	Are important entities and relationships defined?
UI/UX coverage	Are screens, states, and interactions described?
Edge cases	Are failures and unusual scenarios considered?
Security/privacy	Are relevant risks addressed?
Testing	Are verification and acceptance criteria provided?
Dependency coherence	Do the chosen technologies make sense together?
Build readiness	Can an AI coding agent reasonably start from this spec?

Don't send Jev:

Score this plan from 0-100.

Instead, use bounded questions.

For example:

Does the implementation plan address all explicit requirements from the user's project description?

YES / NO

And:

How complete is the implementation specification?

- A. Critical requirements missing
- B. Significant gaps
- C. Minor gaps
- D. Implementation ready

Or:

Which best describes the testing strategy?

missing
minimal
adequate
comprehensive

Then **your own code** converts those answers into your 0–100 Forge Score.

That is important because your research specifically warns that Jev's rubric scores shouldn't be treated as reliable exact measurements; arithmetic and deterministic calculations belong in code. Pasted markdown

Phase 0 deliverable

Before moving forward, we should have a document like:

Evaluation Matrix v0.1

10 categories
~30 individual checks
weights
critical-failure rules
pass threshold
optimization instructions

I would consider **this the next thing we design together**.

Phase 1 — Build the ugly vertical slice

Do not start by designing a beautiful interface.

Build an ugly page that proves:

input → Claude → Jev → Claude → output

I recommend making dick123 a **web application first**, not an iPhone app.

My suggested MVP stack:

Frontend
Next.js
TypeScript
Tailwind

Backend
Next.js server routes

Database
PostgreSQL
Supabase

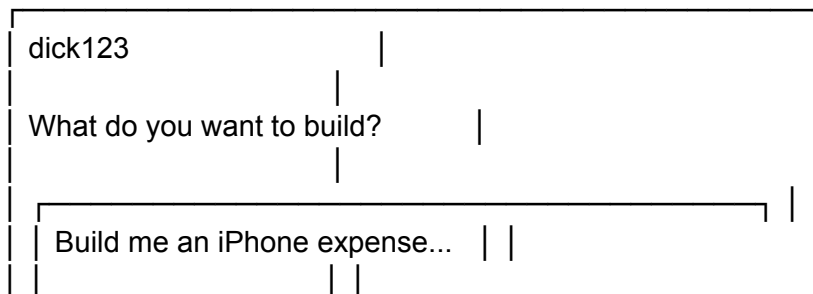
AI
Anthropic API → generation/revision
Jev API → evaluation

Hosting
Vercel

Version control
GitHub

You could replace pieces later. The architecture should deliberately keep Jev behind an interface because it is a hosted closed model and versions/rate limits can change. Your research recommends pinning versions and maintaining provider abstraction/fallbacks. Pasted markdown

The first UI can literally be:





Then display:

INITIAL PLAN

Claude generated:

...

[Evaluate]

Then:

BUILD READINESS

67 / 100

Requirements	92
Architecture	80
Data Model	74
Edge Cases	41
Testing	36

Critical issues:

- Missing acceptance tests
- Error states undefined
- Offline conflicts unspecified

[Optimize]

And finally:

VERSION 2

Build Readiness

91 / 100

+24 improvement

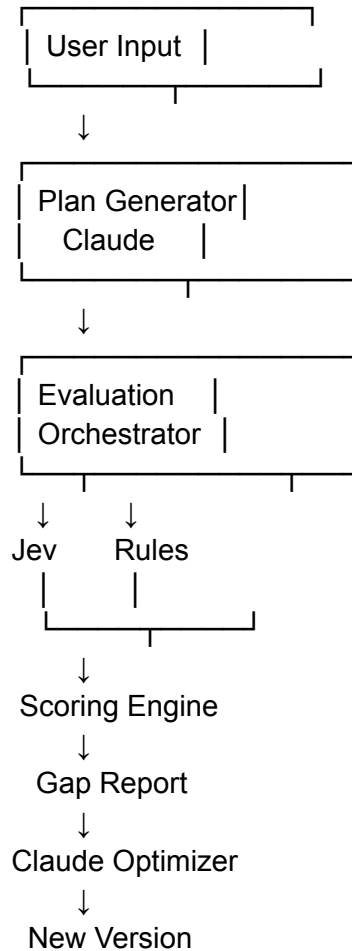
[Copy Final Spec]

That's enough for your first prototype.

Phase 2 — Build the evaluation engine properly

Once the loop works, separate it into components.

Conceptually:



I'd create these internal services:

PlanGenerator
Evaluator
ScoreEngine
Optimizer
VersionManager

And very importantly:

EvaluationProvider

So the rest of dick123 doesn't know whether the evaluator is Jev.

Something roughly conceptually like:

```
interface EvaluationProvider {  
  evaluate(  
    project: ProjectInput,  
    specification: string,  
    rubric: EvaluationRubric  
  ): Promise<EvaluationResult>  
}
```

Then:

JevEvaluationProvider

implements it.

Later you could experiment with:

ClaudeEvaluator
OpenAIEvaluator
LocalEvaluator
CustomClassifier

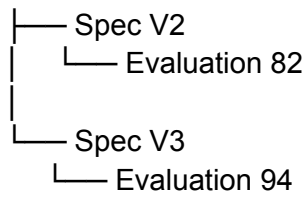
without rewriting your application.

Phase 3 — Add versioning and the optimization loop

Now dick123 starts becoming a real product.

Every project should contain versions:

```
Project  
├── Original Request  
│  
├── Spec V1  
│   └── Evaluation 61  
│  
└──
```



Store the evaluations too.

A minimal database could eventually look like:

users

projects

- id
- user_id
- title
- original_prompt
- created_at

spec_versions

- id
- project_id
- version
- content
- model
- created_at

evaluations

- id
- spec_version_id
- evaluator_model
- rubric_version
- overall_score
- result_json

evaluation_items

- id
- evaluation_id
- criterion
- result
- confidence

optimization_runs

- id
- source_version

- output_version
- optimizer_model

The **rubric version** field is extremely important.

Your research notes that Jev results can depend on question wording. Pasted markdown

Therefore:

testing-completeness-v1

and

testing-completeness-v2

should be treated as different evaluator versions.

Never silently modify a rubric.

Phase 4 — Make the optimizer intelligent

Don't simply tell Claude:

Make this better.

Give it structured failures.

For example:

ORIGINAL REQUEST

...

CURRENT SPECIFICATION

...

EVALUATION SUMMARY

Overall readiness: 72

Critical:

- Authentication error handling missing.
- Data migration strategy missing.

Weak:

- Testing: 43
- Edge cases: 57
- Implementation specificity: 64

Strong:

- Requirements: 94
- Architecture: 91

REVISION INSTRUCTIONS

Preserve requirements that already score well.

Explicitly resolve each critical and weak criterion.

Do not add unrelated features.

Convert vague instructions into testable implementation requirements.

Return the complete replacement specification.

That should give you much better iteration than asking a model to arbitrarily rewrite everything.

You can eventually set rules such as:

Score < 70

→ major rewrite

70–84

→ targeted optimization

85–94

→ fix weak categories only

95+

→ stop

I'd also put a hard iteration limit on it:

maximum 3 optimization passes

Otherwise the system can endlessly rewrite text without meaningfully improving it.

Phase 5 — Build the actual product interface

Only after the engine works should you invest heavily in UI.

I imagine the main screen eventually looking something like:

dick123	Project: Expense App
---------	----------------------

BUILD READINESS

92



Version

V1 63 → V2 81 → V3 92

3 REMAINING ISSUES

Testing

Offline migration behavior doesn't have an acceptance test.

Edge Cases

Network interruption during sync is undefined.

Security

Token storage strategy is unspecified.

[Optimize Again]

The visual experience should feel less like ChatGPT and more like a combination of:

code review

+

CI/CD check

+

project specification tool

That's an important differentiation.

Phase 6 — Build the thing that determines whether this is actually a company

This phase is more important than polishing the UI.

You need to determine whether:

Higher dick123 score



actually means



Better AI-generated software

Your research makes exactly this distinction in another context: a semantic probability or score is not automatically a real-world outcome prediction. Pasted markdown

So eventually run experiments.

Take perhaps 30 small projects.

For every project:

Original prompt



coding agent



Result A

versus:

Original prompt



dick123



optimized specification



same coding agent



Result B

Then measure things like:

Did it compile?

Did tests pass?

How many corrections were required?

How many requirements were missed?

How many agent follow-up prompts were required?

How long did implementation take?

How many regressions appeared?

That is where dick123 becomes interesting.

Because eventually you might discover something like:

Specs scoring >90

required 48% fewer corrective prompts

than specs scoring <70.

That is hypothetical right now.

But **that** is the kind of evidence worth building toward.

Phase 7 — Private beta

Once you've shown the score means something, give it to developers.

Not thousands.

Start with perhaps:

10–25 developers

Watch them use it.

Find out:

- Do they understand the score?
- Do they care about seeing individual failures?
- Do they want a final prompt or a full `.md` specification?
- Are they building with Claude Code, Cursor, Codex, etc.?
- Do they run optimization once or repeatedly?
- What kinds of projects break your evaluator?

At this point you'll probably discover that parts of our original scoring system are wrong.

That's good.

Update the rubric based on evidence.

Phase 8 — Agent integrations

Only after the basic experience works would I add:

Export to Claude Code
Export to Cursor
Export to Codex
GitHub repository context
Existing-codebase analysis
Project rules
Architecture files
PRD import

This is where dick123 can move from:

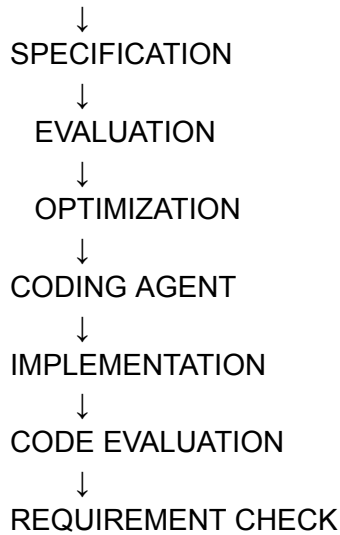
prompt optimizer

toward:

planning and verification layer for AI software development.

Eventually the pipeline becomes:

```
graph TD
  IDEA[IDEA] --> RE[REQUIREMENTS ENGINE]
  RE --> AR[ARCHITECTURE ENGINE]
```



That's potentially a much larger product.

Phase 9 — The long-term moat

Don't make your moat:

We use Jev.

Jev should just be one engine.

The valuable asset could eventually become your dataset:

Project description

+

Specification

+

Evaluation

+

Identified weaknesses

+

Optimized specification

+

Actual implementation

+

Implementation failures

+

Final outcome

Over enough projects, you learn which planning deficiencies actually cause AI coding agents to fail.

That's much harder to reproduce than simply calling an API.

Your attached research makes a similar architectural recommendation: keep the underlying model replaceable and put the durable value into the workflow, evaluation history, thresholds, and outcome evidence. Pasted markdown

Where I would start today

Don't open Cursor and say:

Build dick123.

We're one layer too early.

I would spend the **next step entirely on Phase 0** and create:

dick123 Evaluation System v0.1

1. Evaluation categories
2. Exact Jev questions
3. Possible answers
4. Weight of every criterion
5. Critical failures
6. Score calculation
7. Pass/fail rules
8. Claude optimization prompt
9. Stop conditions
10. Expected JSON schemas

Once that's finished, **then** we give Cursor/Claude a master prompt to build the first vertical slice.

That's the order I'd follow:

design the evaluator → build the smallest working loop → test whether it improves outputs → build the polished product around it.